

REPORT TECNICO AVANZATO: SICUREZZA APPLICATIVA E SECURE CODING

Valutazione delle Vulnerabilità, Analisi del Codice e Conformità ai Pilastri del Secure Coding

1. MODULO: CRITTOGRAFIA E GESTIONE SEGRETI (Cryptography)

Questo modulo valuta la protezione dei dati sensibili, l'implementazione degli algoritmi e la gestione delle chiavi crittografiche.

Livello LOW: Crittografia "Fai-da-te" (XOR)

Il Codice Vulnerabile:

```
PHP
function xor_this($cleartext, $key) {
    $outText = "";
    for($i=0; $i<strlen($cleartext);) {
        for($j=0; ($j<strlen($key) && $i<strlen($cleartext)); $j++, $i++) {
            $outText .= $cleartext[$i] ^ $key[$j]; // ERRORE: Operazione logica XOR
        }
    }
    return $outText;
}
$key = "wachtwoord"; // ERRORE: Chiave hardcodata
```

- **Impatto e Vulnerabilità:** L'uso dell'operatore XOR (^) non è crittografia, ma un banale offuscamento. Poiché lo XOR è perfettamente simmetrico e reversibile, un attaccante a cui sia nota anche solo una piccola parte del testo originale (Known-Plaintext Attack) può calcolare matematicamente la chiave ed esfiltrare l'intero database. Inoltre, la chiave è scritta in chiaro.
- **Pilastri Violati:** **Pilastro 6** (Mai inventare algoritmi crittografici) e **Pilastro 9** (Protezione dei segreti).

Livello MEDIUM: Cifrari Obsoleti (AES-ECB)

Il Codice Vulnerabile:

PHP

```
function encrypt ($plaintext, $key) {  
    // ERRORE: Uso della modalità ECB (Electronic Codebook)  
    $e = openssl_encrypt($plaintext, 'aes-128-ecb', $key, OPENSSL_PKCS1_PADDING);  
    return $e;  
}
```

- **Impatto e Vulnerabilità:** La modalità ECB cifra ogni blocco di dati (di 16 byte) in modo totalmente indipendente, senza usare un Vettore di Inizializzazione (IV). Questo permette a un attaccante di manipolare il token facendo "taglia e incolla" dei blocchi cifrati (Block-swapping). L'attaccante può letteralmente tagliare il blocco che contiene `"level": "user"` e incollarci un blocco (rubato altrove) con `"level": "admin"`, elevando i propri privilegi.
- **Pilastri Violati: Pilastro 6** (Uso di standard obsoleti).

● Livello HIGH: Padding Oracle Attack ed Error Leakage

Il Codice Vulnerabile:

PHP

```
define ("IV", "1234567812345678"); // ERRORE: IV Statico e non casuale  
try {  
    $d = decrypt ($ciphertext, $iv);  
} catch (Exception $exp) {  
    $ret = array (  
        "status" => 526,  
        "extra" => $exp->getMessage() // ERRORE: Information Disclosure  
    );  
}
```

- **Impatto e Vulnerabilità:** L'applicazione rivela all'utente l'eccezione crittografica interna di OpenSSL (`$exp->getMessage()`). Il server funge così da "Oracolo": l'attaccante invia migliaia di token alterando l'IV un byte alla volta e osserva gli errori restituiti. Sfruttando le differenze tra gli errori di "padding non valido" e gli altri errori, decifra l'intero token byte per byte senza avere la chiave.
- **Pilastri Violati: Pilastro 7** (Gestione sicura degli errori) e **Pilastro 6**.

● Livello IMPOSSIBLE: L'Illusione della Perfezione

Il Codice Vulnerabile:

PHP

```
define ("KEY", "rainbowclimbinghigh"); // ERRORE FATALE: Chiave in chiaro nel codice  
define ("ALGO", "aes-256-gcm");
```

- **Impatto e Vulnerabilità:** Anche se l'algoritmo GCM è inattaccabile matematicamente (poiché possiede un tag di autenticazione che blocca le manipolazioni), il sistema è del tutto insicuro. La chiave madre è "hardcodata". Chiunque acceda al file sorgente (es. tramite un LFI o accesso al repository Git) ottiene la chiave e compromette irrimediabilmente l'applicazione.
- **Pilastri Violati: Pilastro 9** (Mancato uso di Secret Management).

🎓 **CONCLUSIONE DEL MODULO CRITTOGRAFIA:** La crittografia non ammette implementazioni "creative" (Pilastro 6). È obbligatorio utilizzare standard industriali autenticati (come AES-256-GCM), generare Vettori di Inizializzazione (IV) casuali per ogni operazione, gestire le chiavi tramite servizi esterni (Vault, AWS Secrets Manager) e non rivelare mai log di errore crittografico agli utenti finali (Pilastro 7).

2. MODULO: INIEZIONE DI COMANDI OS (Command Injection)

Questo modulo dimostra come l'interazione diretta con la shell di sistema possa portare alla compromissione totale dell'host (Remote Code Execution).

● Livello LOW: Esecuzione Diretta

Il Codice Vulnerabile:

```
PHP
$target = trim($_REQUEST['ip']);
// ERRORE: Input passato direttamente alla shell
$cmd = shell_exec('ping -c 4 ' . $target );
```

- **Impatto e Vulnerabilità:** Il server accetta una stringa e la appende a un comando Bash/CMD. Un attaccante inietta metacaratteri come il punto e virgola (;). Inviando `127.0.0.1; cat /etc/passwd`, il server esegue prima il ping, e subito dopo il comando malevolo, cedendo il controllo del server all'attaccante.
- **Pilastri Violati: Pilastro 1** (Fiducia totale nell'input).

● Livello HIGH: La Fallacia delle Deny-List

Il Codice Vulnerabile:

```
PHP
$substitutions = array( '&&' => "&", ';' => ";", '|' => "|", '-' => "-" );
```

```
$target = str_replace( array_keys( $substitutions ), $substitutions, $target ); // ERRORE: Deny-List  
$cmd = shell_exec( 'ping -c 4 ' . $target );
```

- **Impatto e Vulnerabilità:** L'uso di una lista nera (Deny-list) per rimuovere i caratteri cattivi fallisce a causa di un errore umano. La stringa da filtrare ' | ' possiede uno spazio. L'attaccante aggira il filtro inviando `127.0.0.1 | whoami` (senza spazi), forzando l'esecuzione del comando malevolo.
- **Pilastri Violati: Pilastro 1** (Mai usare Deny-List).

● Livello IMPOSSIBLE: Allow-List e Ricostruzione

Il Codice Sicuro:

PHP

```
$octet = explode( ".", $target );  
// CORRETTO: Validazione tramite Allow-list (solo numeri consentiti)  
if( is_numeric( $octet[0] ) && is_numeric( $octet[1] ) && is_numeric( $octet[2] ) &&  
is_numeric( $octet[3] ) && sizeof( $octet ) == 4 ) {  
    $target = $octet[0] . '.' . $octet[1] . '.' . $octet[2] . '.' . $octet[3]; // Ricostruzione del dato  
    $cmd = shell_exec( 'ping -c 4 ' . $target );  
}
```

- **Impatto e Meccanica:** Il codice distrugge l'input, analizza i 4 blocchi e verifica che siano *esclusivamente* numeri. Simboli come | o ; fanno fallire il controllo `is_numeric`. Superato il test, l'IP viene ricostruito da zero e passato alla shell in totale sicurezza.

🎓 **CONCLUSIONE DEL MODULO COMMAND INJECTION:** Le funzioni come `shell_exec` andrebbero evitate in favore di librerie native (es. estensioni cURL). Se inevitabili, la difesa si basa esclusivamente sull'**Allow-List** (definire cosa è permesso) e sulla ricostruzione del dato, supportate dall'uso di funzioni native di escaping del sistema operativo (`escapeshellarg`).

3. MODULO: SQL INJECTION (SQLi)

La SQL Injection avviene quando i dati non sanitizzati alterano la logica di interrogazione del database.

● Livello LOW/MEDIUM: Concatenazione Diretta

Il Codice Vulnerabile:

PHP

```
$id = $_REQUEST['id'];
```

```
// ERRORE: Variabile concatenata direttamente nella query
```

```
$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
```

- **Impatto e Vulnerabilità:** Inserendo `' OR '1'='1'`, la query diviene un'affermazione sempre vera. Il database ignorerà il filtro sull'ID e restituirà all'attaccante l'intero contenuto della tabella, inclusi i dati degli altri utenti. Anche l'uso parziale di `mysqli_real_escape_string` fallisce se la query non usa gli apici attorno alla variabile numerica (Livello Medium).
- **Pilastri Violati: Pilastro 1** (Zero Trust) e **Pilastro 3** (Sanificazione).

● Livello HIGH: Second-Order SQLi

Il Codice Vulnerabile:

PHP

```
$id = $_SESSION['id']; // ERRORE: Fiducia cieca nei dati di sessione
```

```
$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id' LIMIT 1;";
```

- **Impatto e Vulnerabilità:** L'attaccante inietta il payload malevolo in un form secondario che lo salva nella Sessione. Quando la pagina principale estrae l'ID dalla sessione per usarlo, il payload si attiva (Second-Order). L'errore è presumere che i dati siano sicuri solo perché provengono dal proprio database/sessione.
- **Pilastri Violati: Pilastro 1** (Zero Trust esteso ovunque).

● Livello IMPOSSIBLE: Prepared Statements

Il Codice Sicuro:

PHP

```
$data = $db->prepare( 'SELECT first_name, last_name FROM users WHERE user_id = (:id) LIMIT 1;' );
```

```
$data->bindParam( ':id', $id, PDO::PARAM_INT ); // CORRETTO: Parametrizzazione
```

```
$data->execute();
```

- **Impatto e Meccanica:** I *Prepared Statements* risolvono il problema alla radice. La struttura della query SQL e i dati forniti dall'utente vengono inviati al motore del database su due canali separati. Qualsiasi apice o comando inserito dall'utente verrà interpretato esclusivamente come stringa innocua.

🎓 **CONCLUSIONE DEL MODULO SQL INJECTION:** La SQLi non si previene fuggendo i caratteri speciali, ma adottando architetture strutturalmente sicure. L'uso universale di **Prepared Statements** (o ORM moderni) è obbligatorio. È inoltre fondamentale applicare il **Pilastro 2 (Minimo Privilegio)**: l'utente del

database usato dall'applicazione non deve mai avere permessi amministrativi (es. `DROP TABLE` o `FILE`).

4. MODULO: FILE UPLOAD

Il caricamento di file è la via più rapida per installare una "Web Shell" e prendere il controllo dell'infrastruttura.

Livello LOW: Salvataggio Diretto e RCE

Il Codice Vulnerabile:

PHP

```
$target_path .= basename( $_FILES[ 'uploaded' ][ 'name' ] );  
// ERRORE: Nessun controllo su tipo o estensione  
move_uploaded_file( $_FILES[ 'uploaded' ][ 'tmp_name' ], $target_path );
```

- **Impatto e Vulnerabilità:** L'attaccante carica uno script `shell.php`. Il server lo salva nella cartella degli upload. Navigando all'URL del file, l'attaccante esegue comandi sul server (Remote Code Execution).
- **Pilastri Violati: Pilastro 1** (Zero Trust sui file).

Livello MEDIUM & HIGH: Magic Bytes e File Poliglotti

Il Codice Vulnerabile:

PHP

```
$uploaded_type = $_FILES[ 'uploaded' ][ 'type' ]; // ERRORE: Dato fornito dal client  
if( $uploaded_type == "image/jpeg" ) { move_uploaded_file(...); }
```

- **Impatto e Vulnerabilità:** Affidarsi all'intestazione `Content-Type` del client (Livello Medium) è inutile, in quanto modificabile via proxy. Affidarsi alla verifica dei "Magic Bytes" tramite `getimagesize()` (Livello High) permette comunque il caricamento di *File Poliglotti*: file ibridi che iniziano con l'intestazione di una vera immagine GIF/JPG, ma che contengono codice PHP nascosto nei commenti EXIF.
- **Pilastri Violati: Pilastro 1 e Pilastro 5** (Difesa debole).

Livello IMPOSSIBLE: Sanificazione Distruttiva

Il Codice Sicuro:

PHP

```
// CORRETTO: Ricostruzione del media  
$img = imagecreatefromjpeg( $uploaded_tmp );
```

```
imagejpeg( $img, $temp_file, 100);
```

- **Impatto e Meccanica:** Il file caricato non viene spostato, ma processato dalla libreria grafica GD. L'immagine viene ricostruita da zero, pixel per pixel. Questo processo distrugge irrimediabilmente qualsiasi payload PHP nascosto al suo interno, rendendo il file totalmente sicuro.

🎓 **CONCLUSIONE DEL MODULO FILE UPLOAD:** Per proteggere gli upload bisogna: 1) Rinominare sempre i file generandone il nome lato server (es. tramite Hash) per prevenire Path Traversal. 2) Distruggere e ricreare il contenuto multimediale. 3) Disabilitare a livello di Web Server (es. tramite `.htaccess`) l'esecuzione di script all'interno della cartella di destinazione.

5. MODULO: CROSS-SITE SCRIPTING (XSS)

L'XSS sfrutta il browser dell'utente (client) per eseguire script iniettati, permettendo il furto di sessioni.

● Reflected e Stored XSS: Mancata Codifica in Output

Il Codice Vulnerabile (Reflected):

PHP

```
$html .= '<pre>Hello ' . $_GET[ 'name' ] . '</pre>'; // ERRORE: Stampa diretta
```

Il Codice Vulnerabile (Stored):

PHP

```
$query = "INSERT INTO guestbook ( comment, name ) VALUES ( '$message', '$name' );";  
// ERRORE: Salvato in chiaro
```

- **Impatto e Vulnerabilità:** Che il dato venga riflesso direttamente dall'URL o salvato permanentemente in un database per essere poi letto, la mancanza di codifica permette al browser di interpretare i tag HTML (`<script>`) come codice eseguibile. Anche rimuovere la parola `<script>` con le regex è inutile, poiché l'attaccante userà tag alternativi come `<img src=x onerror=alert(1)>`.
- **Pilastri Violati: Pilastro 3** (Mancata Codifica dell'Output).

● DOM-Based XSS: L'Errore Client-Side

Il Codice Vulnerabile (JS):

JavaScript

```
var lang = document.location.href.substring(document.location.href.indexOf("default=")+8);
// ERRORE: Uso di un Sink pericoloso
document.write("<option value='" + lang + "'>" + decodeURI(lang) + "</option>");
```

- **Impatto e Vulnerabilità:** Il server PHP non ha colpe. Il JavaScript legge un dato dall'URL (Source) e lo scrive nel DOM tramite `document.write` (Sink). Un attaccante invia il payload dopo il cancelletto `#` (`?default=Eng#<script>alert(1)</script>`). Il server ignora tutto ciò che segue il `#`, ma il browser lo legge e lo esegue, bypassando ogni firewall o controllo server-side.

● Livello IMPOSSIBLE: Output Encoding

Il Codice Sicuro:

PHP

```
$name = htmlspecialchars( $_GET[ 'name' ] ); // CORRETTO: Codifica entità HTML
```

- **Impatto e Meccanica:** La funzione trasforma i caratteri pericolosi (`<`, `>`, `"`) in entità HTML (`<`, `>`, `"`). Il browser le renderizza a video come semplice testo visivo, disinnescando l'esecuzione del codice.

🎓 **CONCLUSIONE DEL MODULO XSS:** La protezione contro l'XSS risiede interamente nel **Pilastro 3**: l'output encoding contestuale prima della renderizzazione. Lato Frontend, bisogna evitare API pericolose come `innerHTML` a favore di `textContent`. Come strato di "Difesa in Profondità" (Pilastro 5), l'implementazione dell'intestazione **Content Security Policy (CSP)** blocca nativamente gli script non autorizzati.

6. MODULO: BROKEN ACCESS CONTROL (BAC) E IDOR

I difetti di autorizzazione permettono agli utenti di accedere a dati non propri (Insecure Direct Object Reference).

● Livello LOW: Fiducia nei Parametri Client

Il Codice Vulnerabile:

PHP

```
$id = $_GET['user_id'];
// ERRORE: Nessun controllo di proprietà sulla risorsa
$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
```


- **Impatto e Vulnerabilità:** L'applicazione estrae l'utente dal DB basandosi solo sul parametro `user_id` fornito nell'URL. L'Utente 1 può cambiare l'URL in `?user_id=2` e visualizzare il profilo privato dell'Utente 2.
- **Pilastri Violati: Pilastro 2** (Principio del Minimo Privilegio).

● Livello IMPOSSIBLE: Access Control Server-Side

Il Codice Sicuro:

PHP

```
$current_user_id = $_SESSION['user_id']; // ID fidato dal server
if ($current_user_id === $id) {
    // Esecuzione query
}
```

- **Impatto e Meccanica:** Il server non si fida dell'URL. Estrae l'identità reale dell'utente dalla Sessione crittograficamente sicura e verifica l'autorizzazione all'accesso prima di eseguire l'operazione sul database.

🎓 **CONCLUSIONE DEL MODULO BAC/IDOR:** L'autorizzazione non deve mai basarsi su parametri manipolabili (URL, form nascosti, Cookie). Il controllo (chi sei e cosa puoi fare) deve avvenire lato backend in fase di esecuzione, verificando i permessi contro l'identità salvata nella variabile di Sessione.

7. MODULO: SESSIONI E CROSS-SITE REQUEST FORGERY (CSRF)

Gestire male le sessioni permette furti di identità o l'esecuzione forzata di azioni.

● CSRF & Weak Sessions: Prevedibilità e Metodi Errati

Il Codice Vulnerabile (Sessioni Deboli):

PHP

```
$cookie_value = time(); // ERRORE: Scarsa entropia (Timestamp)
setcookie("dvwaSession", $cookie_value);
```

Il Codice Vulnerabile (CSRF):

PHP

```
if( isset( $_GET[ 'Change' ] ) ) { // ERRORE: Modifica di stato tramite GET
    $pass_new = $_GET[ 'password_new' ];
```

```
$insert = "UPDATE `users` SET password = '$pass_new' WHERE user =  
'$current_user';";  
}
```

- **Impatto e Vulnerabilità:** Usare il timestamp per generare l'ID di sessione permette a un attaccante di indovinarlo matematicamente (Session Hijacking). Nel CSRF, il cambio password avviene tramite richiesta **GET** priva di protezioni. L'attaccante nasconde l'URL in un tag **** in un altro sito; se la vittima visita il sito, il browser invierà la richiesta al server vulnerabile allegando il cookie di sessione, cambiando la password a insaputa della vittima.
- **Pilastri Violati: Pilastro 4** (Gestione Autenticazione e Sessioni).

● Livello IMPOSSIBLE: Token Anti-CSRF, Entropia e Flag

Il Codice Sicuro:

PHP

// Sessioni sicure:

```
$cookie_value = sha1(mt_rand() . time() . "Impossible"); // Alta entropia  
setcookie("dvwaSession", $cookie_value, time()+3600, "/", $_SERVER['HTTP_HOST'], true,  
true); // Flag HttpOnly e Secure
```

// CSRF sicuro:

```
checkToken( $_REQUEST['user_token'], $_SESSION['session_token'] ); // Anti-CSRF Token  
$pass_curr = $_GET['password_current']; // Re-Autenticazione
```

- **Impatto e Meccanica:** L'uso combinato di CSPRNG (numeri pseudo-casuali crittografici) rende i cookie inindovinabili. I flag **Secure** e **HttpOnly** proteggono il cookie rispettivamente da attacchi Man-in-the-Middle e XSS. Per il CSRF, la richiesta di un Token nascosto e della *password corrente* neutralizza l'attacco, in quanto l'attaccante non può falsificare parametri che non conosce.

🎓 **CONCLUSIONE DEL MODULO SESSIONI/CSRF:** I Token di sessione devono essere generati dal motore nativo del framework. Operazioni sensibili devono utilizzare metodi **POST/PUT**, implementare il *Synchronizer Token Pattern* (Anti-CSRF) e, per operazioni distruttive, imporre una verifica dell'identità (Re-Autenticazione o MFA).

8. CONCLUSIONE GENERALE DEL REPORT

L'auditing completo del codice sorgente dimostra che l'enorme maggioranza degli incidenti di sicurezza deriva dall'infrazione del **Pilastro 1: Mai fidarsi dei dati provenienti dall'esterno (Zero Trust)**.

Approcci difensivi basati su "Deny-List", controlli demandati al Frontend (JavaScript, campi nascosti), e assenza di codifica in Output rappresentano un fallimento architetturale. La vera Sicurezza Applicativa (Secure Coding) deve essere *strutturale*, integrando di default **Prepared Statements**, controlli di Autorizzazione severi sul backend e l'uso rigoroso di protocolli e algoritmi crittografici standardizzati, garantendo così una vera **Difesa in Profondità (Pilastro 5)**.